3) Une classe **RamadanServer** qui instancie le service distant, démarre le RMI Registry, enregistre le service sous le nom: **RamadanTrendsService**.

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
public class RamadanServer {
  public static void main(String[] args) {
    try {
      // Création du service
.....
      // Démarrage du registre RMI (port par défaut 1099)
.....
      // Enregistrement du service
.....
      System.out.println("Serveur RMI Ramadan démarré avec succès...");
    } catch (Exception e) {
      e.printStackTrace();
    }
  }
}
```

4) une classe **RamadanClient** qui se connecte au registre RMI, récupère le service distant, envoie un tableau représentant les ventes journalières (10 jours avant Ramadan) et affiche la moyenne des ventes, le jour du pic de ventes ainsi que la tendance globale d'achat.

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
public class RamadanClient {
  public static void main(String[] args) {
    try {
      // Connexion au registre RMI
.....
```


555F

AVIS IMPORTANT AUX ETUDIANTS

1. Chacune des feuilles de votre copie doit comporter une étiquette code à barres placée à l'endroit indiqué «coller ici votre code à barres».
2. Une copie d'examen comporte une seule «feuille principale» et des «feuilles suites». Sur chacune de vos feuilles, le code à barres est obligatoire.
3. Cette feuille d'examen est strictement personnelle. Elle ne doit comporter aucun signe distinctif. Elle doit être écrite en noir et/ou bleu.
4. Le non respect de l'une de ces recommandations peut faire attribuer la note ZERO à l'épreuve.

NOTE

Coller ici votre code à barre

00	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20

00	25	50	75

Institut Supérieur d'Informatique et de Mathématiques de Monastir

ISIMM

Examen – S1 – 2025/2026

Filière : *3ème LI*	**Matière :** Développement d'applications réparties	**Enseignant : Marwa Thabet**
Date : 06/ 01 / 2026		**Documents autorisés : Non**
Durée de l'examen : 1h30	Régime d'évaluation : **Mixte**	**Nombre de pages : 10**

Nom, prénom et signature de l'enseignant correcteur	Nombre de copies

Exercice 1

À l'approche du mois de Ramadan, les grandes surfaces et les commerces souhaitent anticiper la demande sur certains produits (produits alimentaires, boissons, pâtisseries, etc.) afin d'optimiser leurs stocks. Pour cela, une application distribuée est mise en place afin d'analyser les ventes journalières enregistrées avant Ramadan et d'en extraire des indicateurs utiles pour la prise de décision. Compléter les parties manquantes de l'application Java RMI composée des éléments suivants :

1) Une interface distante nommée **RamadanTrendsService**. Elle déclare trois méthodes:
   - **calculerMoyenneVentes**: calcule et retourne la moyenne des ventes journalières.
   - **jourPicVente**: retourne le numéro du jour où les ventes sont maximales.
   - **tendanceAchat**: analyse l'évolution des ventes et retourne : "Hausse" si les ventes sont globalement en augmentation, "Baisse" si les ventes sont globalement en diminution et "Stable" sinon.

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface RamadanTrendsService extends  .....{

  .....

  .....

  .....
}
```

2) Une classe **RamadanTrendsServiceImpl** qui implémente l'interface distante.

```
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;
public class RamadanTrendsServiceImpl extends  .....
.....
{
  protected RamadanTrendsServiceImpl() throws  .....{
    super();
}
  @Override
  public double calculerMoyenneVentes  (int[] ventes) throws  ..... {
```



```
    int somme = 0;
    for (int v : ventes) {
      somme += v;
    }
    .....
}
@Override
public ..... ..... (int[] ventes) throws .....  {
    int max = ventes[0];
    int index = 0;
    for (int i = 1; i < ventes.length; i++) {
      if (ventes[i] > max) {
        max = ventes[i];
        index = i;
      }
    }
    // +1 car les jours commencent à 1
    return index + 1;
  }
  @Override
  public ..... ..... (int[] ventes) throws .....  {
    boolean hausse = true;
    boolean baisse = true;
    for (int i = 1; i < ventes.length; i++) {
      if (ventes[i] < ventes[i - 1]) {
        hausse = false;  }
      if (ventes[i] > ventes[i - 1]) {
        baisse = false; } }
    if (hausse) {
      return "Hausse";
    } else if (baisse) {
      return "Baisse";
    } else {
      return "Stable";
    }
  }
}
```

Coller ici votre
code à barre

SUITE

c) Fuite de ressources
d) Blocage du ServerSocket

13. Que fait la ligne suivante dans le code Serveur ?

```
UnicastRemoteObject . exportObject ( obj , 0) ;
```

a) Exporte l'objet pour accepter des connexions sur un port choisi par le système
b) Enregistre l'objet dans le registre RMI
c) Sérialise l'objet pour transmission
d) Crée un stub pour le client

14. Considérez ce code serveur TCP :

```
ServerSocket se = new ServerSocket ( mp );
Socket sckServ = se . accept () ;
```

Que se passe-t-il si le port mp est déjà utilisé par une autre application ?
a) L'exception AlreadyBoundException est levée immédiatement
b) L'exception BindException est levée lors de la création du ServerSocket
c) Le nouveau serveur partage le port avec l'application existante
d) Le système d'exploitation attribue automatiquement un port disponible

15. Analysez ce code client :

```
Socket sck = new Socket (mi , mp ) ;
```

Que se passe-t-il si l'adresse mi est valide mais qu'aucun serveur n'écoute sur le port mp ?
a) Une ConnectException est levée après le timeout par défaut
b) Le socket se connecte automatiquement au port 80
c) Le constructeur retourne un socket non connecté
d) Une UnknownHostException est levée immédiatement

16. Quel est l'équivalent UDP de accept() ?
a) connect()
b) receive()
c) bind()
d) Aucun

17. Que faut-il impérativement spécifier lors de la création d'un DatagramPacket pour la réception en Java ?
a) L'adresse IP et le port du client émetteur
b) Données, Taille de données , adresse IP récepteur, port récepteur
c) Données, Taille de données , adresse IP émetteur, port émetteur
d) L'adresse IP et le port du serveur récepteur

18. L'IDL permet de décrire les services distants de manière indépendante du langage de programmation.
a) Vrai, car c'est un langage universel interprété de la même façon par tous les compilateurs.
b) Vrai, car il offre une description neutre qui peut être mappée vers différents langages via une compilation spécifique.
c) Vrai, car il utilise une syntaxe identique à Java qui est le langage le plus répandu.
d) Vrai, car il impose une représentation binaire unique pour tous les langages.


```
        // Récupération du service distant
        RamadanTrendsService service = .....

        // Données de ventes (10 jours avant Ramadan)
        int[] ventes = {120, 130, 150, 160, 180, 200, 210, 230, 250, 300};
        // Appels distants
        double moyenne = .....

        int jourPic = .....

        String tendance = .....

        // Affichage des résultats
        System.out.println("Analyse des ventes avant Ramadan");
        System.out.println("Moyenne des ventes : " + moyenne);
        System.out.println("Jour du pic de ventes : Jour " + jourPic);
        System.out.println("Tendance d'achat : " + tendance);
      } catch (Exception e) {
        e.printStackTrace();
      }
    }
}
```

5) Quelles sont les étapes de lancement de cette application ?

.....
.....
.....
.....
.....
.....
.....
.....

Exercice 2

Compléter le tableau ci-dessous en inscrivant la lettre correspondante à votre réponse pour chaque question du QCM.

Tableau des réponses									
1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35					

1. Pourquoi un serveur multi-thread peut-il gérer plusieurs clients simultanément ?
   a) Car chaque thread a son propre socket d'écoute.
   b) Car le système d'exploitation multiplexe les flux.
   c) Car les threads partagent la même connexion TCP.
   d) Car chaque thread gère une connexion client distincte.
2. Quel est le rôle de l'IDL dans la RPC ?
   a) Chiffrer les données échangées.
   b) Gérer les threads côté serveur.
   c) Générer le code client et serveur dans un langage spécifique.
   d) Définir les interfaces de manière indépendante du langage.
3. Que retourne ServerSocket.accept() ?
   a) Un thread dédié à la communication avec le client.
   b) Un flux d'entrée directement utilisable pour lire les données.
   c) Un objet ServerSocket pour gérer plusieurs clients.
   d) Un objet Socket représentant la connexion avec le client.
4. Quelle est la différence entre PrintWriter(out) et PrintWriter(out, true) ?
   a) Le deuxième paramètre active la compression des données.
   b) Le deuxième paramètre permet d'écrire en binaire plutôt qu'en texte.
   c) Le deuxième paramètre désactive le buffer pour un envoi immédiat.
   d) Le deuxième paramètre active l'auto-flush après chaque appel à println().
5. Par défaut, le registre RMI (rmiregistry) écoute sur :
   a) Le port 1099
   b) Le port 1909
   c) Le port 1990
   d) Le port 1999

6. Analysez ce code du client UDP :

```
int port = 5123;
int port1 = 2341;
DatagramSocket socket = new DatagramSocket () ;
DatagramSocket socketRecu = new DatagramSocket ( port1 );
```

Pourquoi utilise-t-on deux sockets différents dans le client ?
   a) Pour pouvoir écouter sur un port spécifique (2341) pour la réponse
   b) Pour améliorer les performances de la communication
   c) Parce que UDP ne permet pas d'utiliser la même socket pour envoyer et recevoir
   d) Toutes les affirmations précédentes sont correctes
7. Lors de l'utilisation de MulticastSocket pour rejoindre un groupe :
   a) Les paquets reçus du groupe sont aussi reçus par d'autres sockets écoutant le même port.
   b) La socket doit être liée à un port spécifique, pas un port éphémère.
   c) joinGroup() nécessite une adresse IP multicast valide (224.0.0.0 à 239.255.255.255).
   d) On peut rejoindre plusieurs groupes avec le même socket.
8. Quelle méthode permet de connaître l'adresse IP du client connecté à un ServerSocket?
   a) ServerSocket.getInetAddress()
   b) Socket.getInetAddress() sur le socket retourné par accept()
   c) Socket.getLocalAddress() sur le socket retourné par accept()
   d) ServerSocket.getRemoteSocketAddress()
9. Lorsqu'un thread serveur TCP ferme le Socket client mais que le client tente encore d'écrire :
   a) Le client réussit à écrire mais les données sont perdues.
   b) Le client reçoit une SocketException : Connection reset au prochain write().
   c) Le système d'exploitation du client force la fermeture de sa socket.
   d) Le serveur reçoit une IOException en lisant le flux d'entrée.
10. Si on lance deux ServerSocket sur le même port mais avec des adresses IP différentes (une sur localhost, une sur l'IP publique) :
   a) C'est impossible car le port est occupé par le premier.
   b) C'est possible si le système supporte la liaison multi-adresses.
   c) Le second lance une BindException même si l'adresse IP est différente.
   d) Les deux partageront la même file de connexions entrantes.
11. En Java, concernant UDP multicast : si plusieurs sockets rejoignent le même groupe multicast, que peut-on affirmer avec certitude ?
   a) Un seul socket reçoit toujours le datagramme
   b) Tous les sockets reçoivent nécessairement le datagramme
   c) Le comportement dépend du système, de la configuration des sockets et de la plateforme d'exécution
   d) Toutes les affirmations précédentes sont correctes
12. Quel est le principal risque si le serveur ne ferme jamais le Socket client ?
   a) Deadlock
   b) Perte de paquets

Coller ici votre
code à barre

SUITE

19. RPC fonctionne uniquement avec des langages orientés objet.
    a) Faux, car il repose sur des sockets bruts, indépendants du langage.
    b) Faux, car c'est un paradigme générique qui peut être implémenté dans divers paradigmes de programmation.
    c) Faux, car il a été abandonné au profit des web services qui sont indépendants du langage.
    d) Faux, car il nécessite uniquement un support de la sérialisation, pas de la POO.
20. Le middleware vise à masquer la complexité du réseau et à automatiser les tâches répétitives liées à la communication distribuée.
    a) Vrai, car il gère automatiquement tous les aspects matériels des serveurs.
    b) Vrai, car il convertit automatiquement tout code en applications distribuées.
    c) Vrai, car il élimine le besoin de tout protocole de communication.
    d) Vrai, car il fournit une abstraction des ressources distribuées et des services communs réutilisables.
21. Que signifie l'acronyme MOM ?
    a) Method Oriented Middleware
    b) Message Oriented Middleware
    c) Message Object Manager
    d) Middleware Oriented Message
22. Quel est le rôle principal d'un MOM ?
    a) Exécuter des méthodes distantes
    b) Gérer des transactions distribuées
    c) Échanger des messages entre applications
    d) Compiler des applications Java
23. Quel package Java contient l'API JMS ?
    a) java.jms
    b) javax.jms
    c) java.messaging
    d) javax.message
24. Combien de types de messages JMS existent selon le cours ?
    a) 3
    b) 4
    c) 5
    d) 6
25. Quelle est la différence principale entre Queue et Topic ?
    a) Queue est synchrone, Topic est asynchrone
    b) Queue est un-à-plusieurs, Topic est un-à-un
    c) Queue est point-à-point, Topic est publication/abonnement
    d) Queue stocke les messages, Topic ne les stocke jamais
26. Pourquoi JMS favorise-t-il le découplage entre applications ?
    a) Parce qu'il utilise JNDI
    b) Parce que l'émetteur et le récepteur n'ont pas besoin d'être actifs en même temps
    c) Parce qu'il impose des interfaces communes
    d) Parce qu'il fonctionne uniquement en mode synchrone
27. Quel est le rôle de JNDI dans JMS ?
    a) Sérialiser les messages
    b) Assurer la sécurité

c) Localiser les ressources JMS (CF, Queue, Topic)
d) Gérer les transactions

28. Quelle méthode permet de créer une session transactionnelle ?
a) createSession(false, AUTO_ACKNOWLEDGE)
b) createSession(true, 0)
c) createSession(true, AUTO_ACKNOWLEDGE)
d) createSession(false, CLIENT_ACKNOWLEDGE)

29. Quelle instruction est indispensable pour commencer à recevoir des messages ?
a) session.commit()
b) producer.send()
c) connection.start()
d) context.lookup()

30. Quel type de message JMS permet de transporter un objet Java sérialisable ?
a) TextMessage
b) MapMessage
c) StreamMessage
d) ObjectMessage

31. Pourquoi la méthode onMessage() doit-elle gérer toutes les exceptions ?
a) Parce qu'elle est transactionnelle
b) Parce qu'elle bloque la session
c) Parce qu'elle empêche les doublons
d) Parce qu'elle est appelée par le fournisseur JMS

32. Que se passe-t-il si une session transactionnelle est fermée sans commit ?
a) Les messages sont envoyés normalement
b) Les messages sont partiellement envoyés
c) Toutes les opérations sont annulées
d) Les messages sont persistés automatiquement

33. Quel est l'impact d'un rollback() sur les messages consommés ?
a) Ils sont définitivement perdus
b) Ils sont ignorés
c) Ils sont délivrés ultérieurement
d) Ils sont envoyés dans une autre file

34. Dans un système critique, quel mode de livraison est le plus approprié ?
a) NON_PERSISTENT
b) PERSISTENT
c) DUPS_OK_ACKNOWLEDGE
d) AUTO_ACKNOWLEDGE

35. Pourquoi utiliser CLIENT_ACKNOWLEDGE plutôt que AUTO_ACKNOWLEDGE ?
a) Pour améliorer les performances
b) Pour garantir que le message n'est acquitté qu'après traitement réussi
c) Pour simplifier le code
d) Pour éviter les doublons